React Without Surprises: Building Predictable Applications

📋 Agenda

- State Management Surprises
- Effects Surprises
- Component Surprises

🏠 Format

For each topic:

1. See the **bad version** running — predict what happens
2. Discuss **why** it happens (tie back to React's mental model)
3. **Refactor** it together
4. Apply what we learned on **exercises!**

*** Feel free to interrupt me at any time for questions or relevant comments**

🤖 Workshop examples are:

- Real-world (realistic) examples
- Fully functional
- Production ready

State Management Surprises

State Management Surprises

🎬 Live demo: Todo app

🔍 Spot the problem

- Multiple `useState` calls hold values that can be **calculated** from other state or props
- Multiple `useEffect` calls that just keep the state "in sync"

> Ask ❓ "Can I compute this from what I already have?" — if yes, don't store it

❌ Why is it a problem

- **Two sources of truth** — stored value can drift from the calculated one
- **Extra re-renders** — effect-based state synchronization triggers another render
- **Dependency arrays** to maintain and keep correct by hand
- **More code** to read, test, and maintain

✅ The Fix

- **Compute / derive** the value directly in the render body
- Wrap in `useMemo` only if the calculation is genuinely expensive

✅ Why this is better

- **Single source of truth** — derived values are always correct by construction
- **Impossible** to have out-of-sync state
- **Better performance** - no extra re-renders
- **Zero dependency arrays** to maintain
- **Less code** — easier to understand, easier to maintain, simplifies debugging

🎯 Rule of thumb

> If you can calculate it, don't store it.

State Management Surprises

🎬 Live demo: Profile form

State Management Surprises

🚀 Exercise: User Cart

State Management Surprises

🎬 Live demo: App Layout

🔍 Spot the problem

- 🕳️ Prop drilling!
- Passing props through **multiple component layers**
- Intermediate components don't use the props — just forward them
- Can couple intermediate components to data they do not use

```
App → Header → HeaderActions → UserInfo → Profile
      ↓ user    ↓ user         ↓ user      ↓ user
```

> Ask ❓ "Do all these intermediate components use the specific props?"
> — if no, consider Context or a store

❌ Why is it a problem

- Intermediate components become coupled to data they neither own nor use
- Adding or renaming a prop means updating every pass-through component
- Component APIs grow with details unrelated to their actual responsibilities
- The connection between the data's owner and its consumer becomes harder to follow
- This is mainly a **maintainability** problem; prop drilling alone does not necessarily hurt performance

✅ The Fix

- Put genuinely shared data in **global state** (Context API or global store)
- Read it where it is needed with a custom hook or store selector
- Keep props for data that belongs to a nearby component relationship

✅ Why this is better

- Intermediate components no longer carry data they don't use
- Consumers declare what shared data they depend on
- Shared state has a clear owner and is easier to change

🎯 Rule of thumb

> Use props for local, explicit data flow; use Context or a store when distant parts of the tree need the same shared state.

State Management Surprises

🚀 Exercise: Appointment Booking

🔑 Key takeaways

1. **Derive, don't store** — if it can be calculated, don't `useState` it
2. **Choose the right data flow** — use props locally and Context or a store for genuinely shared state

Effects Surprises

Effects Surprises

🎬 Live demo: Message board

🔍 Spot the problem

- A user action happens, but we handle the business logic and state updates in effects
- Often, we split the business logic and state updates between two places: the event handler and the `useEffect`
- This not only reduce the accessibility of the component, but also splits interaction logic across event handlers and effects, making behavior harder to trace

❌ Why is it a problem

- The effect runs **after** the interaction, so the cause and the result live in different places
- The flow becomes indirect: event → state update → re-render → effect → more state updates
- Extra renders and effects that can re-run for unrelated reasons (dependency changes, Strict Mode)
- The event handler, if there is one, tells only half the story

✅ The Fix

- Move the business logic and state updates into the event handler (`onClick`, `onSubmit`, etc.)
- Use semantic controls for interactions: `<button>` and `<form>`, not clickable `<div>`s
- Use effects to synchronize with external systems; keep event-specific logic in event handlers

✅ Why this is better

- One place to look: the handler shows exactly what an interaction does
- Less state and fewer effects (and dependency arrays) to keep track of
- Interaction logic runs in response to the event, not later when dependencies change
- Semantic controls provide built-in keyboard and screen reader behavior; labels and other accessibility details still matter

🎯 Rule of thumb

> Events for what users do, effects for external side-effects.

Effects Surprises

🎬 Live demo: User search

🔍 Spot the problem

- An HTTP request is a side effect
- Fetching in `useEffect` means hand-writing the same boilerplate every time and quickly becomes complicated
- It gets more complicated when you add loading and error states, abort signals, and so on
- And you still don't get retries, mutations, revalidation, or client-side caching

❌ Why is it a problem

- The same fetch logic can make components difficult to maintain and reason about
- Easy to get wrong: stale or out-of-order data; Strict Mode's extra Effect cycle in development can expose missing cleanup and duplicate requests
- Every new feature (retries, mutations, revalidation, caching) adds even more code and complexity

✅ The Fix

- Extract repeated fetch logic into a custom hook
- Use a query library for client-side data when you need caching, retries, or request deduplication
- This makes components easier to maintain and reason about

✅ Why this is better

- The library tracks loading, errors, and data states while reducing boilerplate
- Caching, request deduplication, retries, and revalidation are built in
- Query keys and caching help keep results associated with the right request
- The component can focus on its business logic and remains easier to maintain

🎯 Rule of thumb

> Don't hand-roll data fetching in effects. Use a query library, or server components if you can.

Effects Surprises

🚀 Exercise: Ticket desk

🔑 Key takeaways

1. **Events for interactions, effects for synchronization** — keep user-triggered logic in event handlers and use effects to synchronize with external systems
2. **Keep data fetching out of component effects** — use a query library, or isolate request lifecycle logic in a custom hook

Component Surprises

Component Surprises

🎬 Live demo: Memoization

🔍 Spot the problem

- Expensive calculations can make the interface feel slow or unresponsive
- Work may repeat on renders even when its inputs have not changed
- New object and function references can prevent memoized components from skipping renders

> Ask ❓ "Did this value change, or did I just create a new reference?"

❌ Why is it a problem

- Expensive synchronous work can block the main thread
- Unnecessary work can make updates feel slower
- Memoization relies on stable inputs; new references can look like changed props
- Adding memoization everywhere can add complexity without improving performance

✅ The Fix

- Use the React Compiler when it is available and enabled
- Otherwise, use React APIs such as `useMemo`, `useCallback`, or `React.memo` to address bottlenecks

✅ Why this is better

- `React.memo` can let a component skip a parent-driven render when its props are unchanged
- Optimization stays focused on actual costs

🎯 Rule of thumb

> Make it correct first; optimize measured work, not every component.

Component Surprises

🎬 Live demo: Rendering priorities & layout

🔍 Spot the problem

- Expensive rendering can compete with typing and other urgent interactions
- Large updates can make navigation feel unresponsive
- Measuring layout after paint can cause visible shifts

> Ask ❓ "Which updates are urgent, and does this measurement need to happen before paint?"

❌ Why is it a problem

- Long rendering work can block the main thread
- Users may see stale results or wait for an interaction to finish
- Post-paint layout corrections can cause visible movement
- Layout effects used unnecessarily can delay painting

✅ The Fix

- Use `useDeferredValue` when expensive content can update after urgent input
- Use `useTransition` for non-urgent updates, and communicate when they are pending
- Use `useLayoutEffect` only when a layout measurement must happen before paint

✅ Why this is better

- Urgent interactions stay responsive while expensive updates catch up
- Pending work can be communicated to users
- Layout is measured before the user sees it, when necessary

🎯 Rule of thumb

> Keep urgent updates urgent; defer expensive rendering, and measure layout before paint only when needed.

Component Surprises

🎬 Live demo: Compound components

🔍 Spot the problem - bad example 1

- Every tab must match a fixed data shape (for example `tabTitle`, `title`, and `content`)
- The component controls the markup for both navigation and content
- Consumers can’t compose arbitrary JSX or rearrange the tab structure

> Ask ❓ "Can consumers customize the UI without changing the component itself?"

🔍 Spot the problem - bad example 2

- Composition gives consumers more control over the markup
- Components with complicated APIs (props, state, etc.) must be wired together correctly to make the tabs work
- This makes the API difficult to use

> Ask ❓ "What should consumers control, and what should the component handle?"

❌ Why is it a problem

- Fixed data shapes can limit how consumers structure and customize the UI
- Repeated coordination logic makes components harder to compose
- Exposing internal details can make APIs harder to use

✅ The Fix

- Use compound components and keep the flexibility of the composition
- Create a very simple API that will hide the complexity of the components
- Give consumers useful control through a clear API

✅ Why this is better

- A small API with flexible markup
- Internal coordination logic stays inside the component
- Easier to integrate and use consistently

🎯 Rule of thumb

> Compound components let consumers control the markup while the parent coordinates shared behavior behind a clear API.

Component Surprises

🚀 Exercise: Product cards

Component Surprises

🎬 Live demo: Suspense

🔍 Spot the problem

- Each widget handles its own loading state and missing data
- Loading and error handling are repeated around otherwise simple UI

> Ask ❓ "Can the loading boundary own this state instead?"

❌ Why is it a problem

- Every component must account for pending and failed requests
- Repeated status checks obscure the UI's main purpose
- One shared loading state can hide which part of the page is waiting

✅ The Fix

- Use Suspense-enabled queries for data that can suspend
- Place Suspense and error boundaries around the parts that should fall back independently

✅ Why this is better

- Components can render with defined data
- Loading and errors are handled at clear boundaries
- Each widget can show its own fallback

🎯 Rule of thumb

> Put loading and error handling at the boundary that matches the user experience.

🔑 Key takeaways

1. **Optimize with evidence** — use the compiler or targeted memoization to address measured costs
2. **Keep interactions responsive** — let non-urgent rendering wait, and measure layout before paint only when needed
3. **Design for composition** — give consumers control over structure while keeping coordination inside the component
4. **Place clear boundaries** — handle loading and errors where the fallback should appear

🙏 That's all folks

Questions / Discussions?